

# Simultaneous Reverse Engineering & Feature Identification with Machine Translation

Hugo Espiritu  
Computer Science  
University of Texas at Dallas  
Dallas, United States  
[Hugo.Espiritu@utdallas.edu](mailto:Hugo.Espiritu@utdallas.edu)

Hailey Gage  
Computer Science  
University of Texas at Dallas  
Dallas, United States  
[Hailey.Gage@utdallas.edu](mailto:Hailey.Gage@utdallas.edu)

Nick Ramos  
Computer Science  
University of Texas at Dallas  
Dallas, United States  
[nor140030@utdallas.edu](mailto:nor140030@utdallas.edu)

Suleyman Olcay Polat  
Applied Computer Science  
Columbus State University  
Columbus, United States  
[Polat\\_suleymanolcay@columbusstate.edu](mailto:Polat_suleymanolcay@columbusstate.edu)

**Abstract**—Neural machine translation models are effective at translating between natural languages and can be modified to accommodate stricter structural and grammatical rules of programming languages. This paper focuses on comparing different machine learning models for finding function starting points in binary files as a first step to create a machine translation model for decompiling.

*Reverse engineering, decompilation, machine learning, Statistical models*

## I. INTRODUCTION

Most reverse engineering tools are inefficient, incomplete, and limited in scope. In particular, disassembly listings have a very low abstraction level and require an analyst to manually map assembly instructions to higher level abstractions. The highly repetitive nature of the listings and the need for meticulous attention to changes in small details make the task difficult for beginners and very tedious for experienced analysts. The amount of code analyzed is often massive and may require quick analysis to keep up with adaptive malware and attacks. This domain is an ideal situation for automated analysis with supervised machine learning. The success of neural machine translation (NMT) models for translation of sequences of natural language text inspired work that modifies NMTs to handle stricter constraints imposed by translation of programming languages. In this paper we attempt to solve function identification (one of the subtopics for reverse engineering) and compare the performance of common machine learning models used on NMTs to find best suited model for reverse engineering.

## II. BACKGROUND

Past work in machine learning for decompilation of binary code has focused on a variety of approaches ranging from statistical machine translation to adapting neural machine translation models (NMTs).

The statistical machine translation techniques largely focus on adapting phrase-based statistical machine translation

models, use programming language grammars and in some cases also incorporate NLP deep learning models such as Word2Vec [1] [2] [3]. An interesting example is the work by Vasilescu et al [4] that implements a tool that uses off-the-shelf statistical machine translation models (MOSES) to recover some of the original names of Javascript programs minified by the popular tool UglifyJS. The main assumption that is leveraged in this work is the highly repetitive nature of software; this intuition is captured with a statistical language model. This language-independent approach performs well when additional tuning of training data (to account for JS syntax) and some post-processing (to manage scoping) is added. The resulting tool of combining this approach with the previously best performing deobfuscator for Javascript, JSNice (uses static analysis), performs significantly better than either of the individual methods.

The neural machine translation model approaches use large code datasets for byte-equivalent decompilation using RNN encoder-decoder models to learn a decompilation function for translating small snippets of machine code (approximately 100 machine code bytes) to C source code [5]. The main contributions of this work include an NMT architecture modified to translate between programming language representations (including compiled machine code), post processing techniques to improve the RNN decompilation, and useful tokenization schemes to support reverse engineering. However, a variety of challenges remain for making the RNN encoder-decoder architecture a viable replacement to traditional decompilers including support for longer snippets of machine code bytes, ensuring that train sets and test sets don't contain the same code snippets, and verifying that the decompiled code is useful to humans. Another approach that uses modified NMTs to translate between a source language and a target language involves a tree-to-tree encoder-decoder architecture evaluated against several state-of-the-art approaches [6]. The contributions of this work include the ability to handle longer code snippets than sequence-to-sequence models that encode token sequences without structural information and useful metrics for estimating the utility of the decompiled

code. The main drawbacks of this approach are the difficulty of generalizing the model to decompile programs longer than the programs used for training the model and the inability to handle an infinite vocabulary set.

In the specific case of Android applications, Bichsel et al [7] implements a tool, DeGuard, that uses probabilistic graphical models (specifically, conditional random fields) for deobfuscation of Android APKs that are obfuscated with the widely used ProGuard obfuscation tool. This work focuses on recovering identifier names and identifying third party libraries by learning from thousands of readily available, non-obfuscated Android applications and has 79.1% recovery of program element names, predicts third-party libraries with accuracy of 91.3% and reports on several examples that suggest that DeGuard can be useful when inspecting malware. The main limitation of DeGuard is the inability to deobfuscate additional data obfuscation techniques that alter data structures, control flow, and common malware obfuscation techniques (eg custom encoding of strings, extensive use of reflection, code reordering using goto statements, etc).

### III. METHODS AND MATERIALS

#### A. Dataset and Data preparation

For our experiments we used a dataset created by Tiffany Bao et al [8]. The dataset consists of consists of 2,200 binaries created with three different compilers, with four different optimization levels, and across two different operating systems. In our experiments we only used lowest optimization level binaries for linux 32 bit operating system. We choose to use lowest optimization level to compare different models on the most basic task first. After selecting the binary files we used binary analysis tool (BAP) to disassemble them and used Byteweight [8] tool to find function starting points . In the current for the dataset we used is divided to 10 fold and for each fold we have binaries with flags for each byte that specifies them as a starting byte for a function or not.

Main problem with the dataset we used is the imbalance of classe. In the current form the ratio of function starts to bytes is 1:1000 . This causes models to classify every byte as a not function starter and still achieve accuracy of 99.9%.

#### B. Support Vector Machine

Support Vector Machines are supervised learning models that work by marking data, that is given by a set of training examples, as belonging to one or the other of two categories and builds a model that assigns new examples to one category or the other. The main advantage to SVM is that they always find the global maximum. They're resistant to overfitting with the help of regularization parameter. However, the main disadvantage is it needs features to be created from data.

For our implementation of SVM, we were able to construct a python script that ran the dataset. We used the "Sklearn" library that contains an implementation of SVM. We encountered a problem when running the SVM because of the dataset being imbalanced. The results were incorrect

because of the classes being imbalanced. . However, we were able to find a solution by finding the optimal separating hyperplane using an SVC for classes that are unbalanced.

#### C. Random Forest

Random Forest generates several different decision trees and ultimately decides on one decision from these trees. Random forest has the added benefit of greater diversity with its tree generation than other tree based algorithms. This is because each tree uses a different subset of features and a random set of training data points, training each tree with this random set of features and training data points. Some of the advantages that Random Forest has over other algorithms is that it is accurate with most data sets, including large sets. Not only this, it runs efficiently even with large datasets. However, one drawback to using this algorithm compared to others is that overfitting can easily occur and, more importantly, it needs features to be created from the data like the Support Vector Machine.

Similarly to our implementation of Support Vector Machine, our Random Forest implementation used the "Sklearn" python package, which also has a random forest implementation. When attempting to implement this algorithm to the dataset, we were unable to account for class imbalances due to time restraints. This lead to inconclusive results.

#### D. Recurrent Neural Net

Recurrent neural network is a class of artificial neural network where connections between nodes form a directed graph along a sequence. The main advantage for RNN is it does not need feature creation and time frames. It can keep information of all the code in the model. However, the main disadvantage is high possibility of overfitting and it requires a long training time with a big dataset.

#### E. Seq2Seq

Seq2Seq is a general-purpose encoder-decoder framework for Tensorflow that can be used for machine translation, text summarization, and more. The main advantage for seq2seq is it does not need feature creation and time frames. It can also keep information of all the code in the model. The main disadvantage of using seq2seq is the high possibility of overfitting and creating more classes than the bytes we have.

### IV. EXPERIMENT

Because of time constraints we only used the first fold of our dataset for training and used rest of the folds for training. For SVM and Random Forest we used sparse matrix encoding to convert our bytes to features we can use on the models. For RNN and Seq2seq we handle the embedding inside the model with a embedding layer.

To counter class imbalance we used class weights on error functions for all of our models.

For our results after running the Random Forest classifier, we had 3,514,047 true negative results, 9,487 false negatives, but 0 true and false positives. The inconclusive results are due to the imbalances in the dataset.

For our results after running the Support Vector Machine, we had 2,590,980 true negatives, 923,067 false positives, 7,244 false negatives, 2,193 true positives, 23% of the function starting points found, and 923,067 bytes miss classified as function starts.

For our results after running Recurrent Neural net, we had 3,514,047 true negatives, 0 false positives, 9,487 false negatives, and 0 true positives. These results were due to it not being modified to handle class imbalance.

For our results after running the Seq2Seq classifier, we had 2,127,520 true negatives, 1,386,527 false positives, 6,242 false negatives, 3,245 true positives, 34% of the function starting points founds, and 1,386,527 bytes miss classified as function starts.

## V. CONCLUSION

This project summarizes our efforts in modifying off-the-shelf machine learning classifiers for the task of function start identification. Our preliminary results demonstrate that more work is needed to reduce the large number of false positives and improve the overall function start identification rate. Viable next steps include tuning the current classifiers and incorporating heuristics used in static analysis to help with preprocessing the binary data to get a competitive function start classification accuracy and expand the reverse engineering feature identification goal to other related tasks.

## VI. REFERENCES

- [1] B. E. T. D. P. Allamanis, Miltiadis and C. Sutton, "A survey of machine learning for big code and naturalness," 2017.
- [2] R. V. Karaivanov, Svetoslav and M. Vechev, "Phrase-based statistical translation of programming languages." 2014.
- [3] N. A. T. Nguyen, Trong Duc and T. N. Nguyen, "Mapping api elements for code migration with vector representations," 2016.
- [4] P. D. Bogdan Vasilescu, Casey Casalnuovo, "Recovering clear, natural identifiers from obfuscated js names," 2017.
- [5] E. S. Deborah S. Katz, Jason Ruchti, "Using recurrent neural networks for decompilation," 2018.
- [6] D. S. Xinyun Chen, Chang Liu, "Tree-to-tree neural networks for program translation," 2018.
- [7] P. T. M. V. Benjamin Bichsel, Veselin Raychev, "Statistical deobfuscation of android applications," 2016.
- [8] Bao, T., Burket, J., Woo, M., Turner, R., & Brumley, D. , " Byteweight: Learning to recognize functions in binary code" 2014.

